

# Paths To Improve VAE

## Loss Function Improvements

Currently uses MSE and KL Divergence to calculate the loss for the encoder and decoder (they are packaged together, so there is only one loss function)

- Don't think it's possible to have two separate loss functions because you can't evaluate it in between the processes (it is an unsupervised machine learning method), so we are stuck with optimizing the two of them at the same time.
1. Implementing a more domain specific loss function (makes it more relevant to the actual input data)
    - SSIM
    - SAM

## Training Mechanism

Currently, the program calculates the loss for both encoder and decoder at the same time, and has the weights propagated backwards. It is trained all at once (as in, for every loss value, backpropagation is performed on both the parts of the VAE)

- Not something I really wanna implement, but it is something that you do for GANs too. You "pause" training for one of the parts (usually encoder because that is easier to learn), allow the other part to catch up, then continue onwards.
  - Prevents model collapse (when one part gets too good at its job, the other part is preventing the other part from learning. This is something seen a lot in adversarial networks (where they are evaluating one another), which is not what's happening here, so I'm not entirely sure on how helpful this would be for a non-adversarial method.
  - To implement it, you would take a "snapshot" of one part (all its weights and biases, use that and don't backpropagate through it (as in, you calculate the gradient, but don't update any values), and allow the other part to catch up.
    - You could use gradient based methods to see if updating both leads to faster loss decreases or updating just one at a time (and you have inertia and you would choose that one for a while)

- Requires 3x the computation, might prevent model collapse, probably is worth it if it starts plateauing too early on